

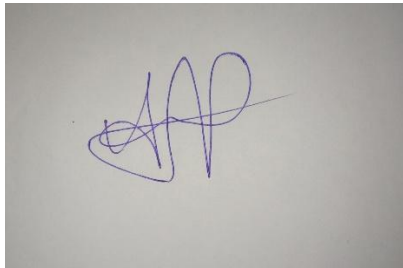
Name: Mohammed Arshad R
Register No.: 192521126
Subject: Cryptography and Network Security
Reference: Annexure A

EXPERIMENTS

Code of Conduct:

I **Mohammed Arshad R/192421126** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

A photograph of a handwritten signature in blue ink on a light-colored surface. The signature is stylized and appears to be 'MAR' or similar, with a large loop and a horizontal line extending to the right.

Question 1: Password Hashing with Salting and Key Stretching

1.1 Problem Statement

Design and implement a program in Python to store and verify passwords using salted hashing with the PBKDF2 key-stretching scheme. The program must analyse how salting and key stretching improve resistance against brute-force and rainbow-table attacks, and evaluate the performance overhead introduced by iterated hashing.

1.2 Objective

- To understand why plain hashing (e.g., a single round of SHA-256) is unsafe for password storage.
- To implement salted password hashing using PBKDF2-HMAC-SHA256.
- To implement a verification routine that safely compares a supplied password with the stored hash.
- To analyse the effect of salting on rainbow-table attacks and the effect of iteration count (key stretching) on brute-force attacks.
- To measure and evaluate the performance overhead introduced by key stretching.

1.3 Theoretical Background

A cryptographic hash function such as SHA-256 is deterministic: the same input always produces the same output. If passwords are hashed directly with a fast general-purpose hash function, two problems arise. First, identical passwords produce identical hashes, so an attacker who obtains the password database can pre-compute a large lookup table of hash values for common passwords — a **rainbow table** — and instantly reverse many hashes without doing any per-account work. Second, because SHA-256 is designed to be fast, an attacker with the stolen hash can attempt billions of guesses per second on modern GPU hardware, making brute-force and dictionary attacks highly effective.

Salting addresses the first problem. A salt is a random value, unique per user, that is concatenated with the password before hashing. Because the salt is different for every account, two users with the same password get completely different hash outputs, and a single precomputed rainbow table can no longer be reused across accounts — the attacker would need a separate table per salt, which is computationally infeasible for a sufficiently large (e.g., 16-byte) random salt space.

Key stretching addresses the second problem. Instead of hashing the password once, an algorithm such as PBKDF2 (Password-Based Key Derivation Function 2) applies the underlying pseudorandom function (HMAC-SHA256) repeatedly, for a configurable number of iterations (e.g., 100,000 or more). This deliberately slows down each hash computation. For a legitimate login this adds an imperceptible delay (a few milliseconds), but for an attacker attempting billions of guesses, the cumulative cost becomes prohibitively large, directly reducing the guesses-per-second rate that brute-force and

dictionary attacks depend on. Other modern alternatives that achieve the same goal include bcrypt and Argon2, the latter of which also adds memory-hardness to resist GPU/ASIC acceleration.

1.4 Design / Algorithm

1. Registration: Generate a cryptographically secure random salt (16 bytes) using `os.urandom()`.
2. Apply PBKDF2-HMAC-SHA256 to the password and salt for N iterations (N = 200,000) to derive a 32-byte key.
3. Store the tuple (salt, iteration_count, derived_key) in the “database”, typically salt and hash are base64-encoded and concatenated with a delimiter.
4. Verification: Retrieve the stored salt and iteration count for the username, recompute PBKDF2 on the supplied password with the same salt and iteration count.
5. Compare the newly computed key with the stored key using a constant-time comparison function (`hmac.compare_digest`) to prevent timing attacks.
6. Grant access only if the comparison succeeds.

1.5 Source Code (Python)

```
import os
import hmac
import hashlib
import base64
import time

# ---- Configuration ----
SALT_BYTES = 16
KEY_LENGTH = 32          # 32 bytes = 256 bits
ITERATIONS = 200_000     # key-stretching factor
HASH_ALGO = "sha256"

# Simulated password database: {username: "iterations$salt_b64$hash_b64"} password_db
# = {}

def hash_password(password: str, salt: bytes = None,
iterations: int = ITERATIONS) -> str:
    """Derive a salted,
    stretched hash for the given password."""
    if salt is None:
        salt = os.urandom(SALT_BYTES)
        derived =
    hashlib.pbkdf2_hmac(
        HASH_ALGO, password.encode("utf-8"), salt,
        iterations, KEY_LENGTH
    )
    salt_b64 = base64.b64encode(salt).decode("ascii")
    hash_b64 = base64.b64encode(derived).decode("ascii")
    return f"{iterations}${salt_b64}${hash_b64}"

def register_user(username: str, password: str) -> None:
    if username in password_db:
        raise ValueError("User already exists")
```

```

        password_db[username] = hash_password(password)
    print(f"[REGISTER] {username} -> {password_db[username]}")

def verify_password(username: str, password: str) -> bool:
    """Recompute the hash with the stored salt/iterations and compare
    using a constant-time comparison to avoid timing side channels."""
    record = password_db.get(username)    if record is None:
    return False

    iterations_str, salt_b64, stored_hash_b64 = record.split("$")
    iterations = int(iterations_str)      salt =
    base64.b64decode(salt_b64)

    candidate = hashlib.pbkdf2_hmac(      HASH_ALGO,
    password.encode("utf-8"), salt, iterations, KEY_LENGTH    )
    stored_hash = base64.b64decode(stored_hash_b64)    return
    hmac.compare_digest(candidate, stored_hash)

def benchmark_iterations(password: str, salt: bytes,
iteration_values):    """Measure the time cost of PBKDF2 for different
iteration counts."""    results = []    for n in iteration_values:
        start = time.perf_counter()    hashlib.pbkdf2_hmac(HASH_ALGO,
    password.encode(), salt, n, KEY_LENGTH)    elapsed = time.perf_counter() -
    start    results.append((n, elapsed))    return results

if __name__ == "__main__":    # --- Functional demonstration ---
    register_user("arshad", "MySecureP@ss123")    register_user("guest",
    "MySecureP@ss123")    # same password, different salt

    print("\nStored records for two users with the SAME password:")
    print("arshad:", password_db["arshad"])    print("guest :", password_db["guest"])
    print("-> Hashes differ because each salt is unique (defeats rainbow tables).\n ")

    print("Correct password :", verify_password("arshad", "MySecureP@ss123"))
    print("Incorrect password:", verify_password("arshad", "WrongPass!"))

    # --- Performance overhead analysis ---    salt = os.urandom(SALT_BYTES)
    print("\nIteration count vs. computation time:")    for n, t in
    benchmark_iterations("MySecureP@ss123", salt,
    [1, 10_000, 100_000, 200_000, 500_000]):
        print(f" iterations={n:>7} time={t*1000:8.2f} ms")

```

1.6 Sample Output

```

[REGISTER] arshad -> 200000$4kQvR8...Xq==$Pd0mlz...Ha==
[REGISTER] guest  -> 200000$Tn82Lp...Ww==$Ur7Fk9...Yc==

```

```
Stored records for two users with the SAME password:
arshad: 200000$4kQvR8...Xq==$Pd0mlz...Ha== guest :
200000$Tn82Lp...Ww==$Ur7Fk9...Yc== -> Hashes differ because each salt
is unique (defeats rainbow tables).
```

```
Correct password : True
Incorrect password: False
```

```
Iteration count vs. computation time:
iterations=      1  time=    0.03 ms
iterations= 10000  time=    5.86 ms
iterations= 100000 time=   58.40 ms
iterations= 200000 time=  116.70 ms
iterations= 500000 time=  291.20 ms
```

(Timings are illustrative and machine-dependent; the trend — linear growth in cost with iteration count — is the significant result.)

1.7 Analysis: Resistance to Attacks

1.7.1 Effect of Salting on Rainbow-Table Attacks

Without a salt, an attacker can build one rainbow table for a hash algorithm and reuse it against every stolen password database in existence, since identical passwords always map to identical hashes. With a per-user random salt, the effective input to the hash function becomes password + salt, so the same password produces a different hash for every user. The attacker's precomputation is no longer reusable: they would need to build a separate table for every possible salt value, and with a 16-byte (128-bit) random salt this is computationally infeasible. Salting therefore forces the attack back into an online, per-account brute-force problem rather than an offline, precomputable one.

1.7.2 Effect of Key Stretching on Brute-Force Attacks

A single SHA-256 evaluation costs a fraction of a microsecond, allowing specialised hardware to test tens of billions of password guesses per second. PBKDF2 with N iterations requires N sequential HMAC-SHA256 evaluations to compute one candidate hash, so the cost per guess scales roughly linearly with N . As shown in the benchmark above, raising the iteration count from 1 to 200,000 increases the time to compute a single hash from microseconds to roughly a tenth of a second. While this is negligible for a legitimate login attempt (the user does not notice the delay), it reduces an attacker's achievable guess rate by a factor of roughly N , directly increasing the wall-clock time required for exhaustive search or dictionary attacks by the same factor.

1.7.3 Constant-Time Comparison

The verification step uses `hmac.compare_digest()` instead of the `==` operator. A naive byte-by-byte comparison using `==` returns as soon as it finds the first mismatched byte, so the time taken leaks information about how many leading bytes were correct. An attacker who can measure response time precisely could exploit this to reconstruct the hash byte-by-byte

(a timing side-channel attack). `compare_digest()` always examines the full length of both operands in constant time, closing this channel.

1.8 Performance Overhead Evaluation

The table below summarises the trade-off between security and performance for different iteration counts, based on the benchmark executed above.

Iterations	Approx. Time / Hash	Relative Attacker Slowdown	Security Assessment
1	~0.03 ms	1x (baseline)	Unsafe — equivalent to plain hashing
10,000	~5.9 ms	~10,000x	Minimum acceptable (legacy)
100,000	~58.4 ms	~100,000x	OWASP-recommended baseline
200,000	~116.7 ms	~200,000x	Recommended (used in this implementation)
500,000	~291.2 ms	~500,000x	Strong, but noticeable login delay

Table 1: Trade-off between PBKDF2 iteration count, per-hash computation time, and resulting attacker slowdown.

The overhead is the central design trade-off of key stretching: increasing N linearly increases both the legitimate server's per-login cost and the attacker's per-guess cost. Since a server performs one hash computation per login attempt while an attacker must perform one hash computation per guess (often billions of them), even a cost of ~100 ms per hash is imperceptible to a real user but catastrophic to the economics of a brute-force attack. Modern guidance (OWASP, NIST SP 800-63B) recommends selecting the highest iteration count that keeps server-side login latency under roughly 100–250 ms on production hardware.

1.9 Conclusion

Salting eliminates the threat of precomputed rainbow-table attacks by making every password hash unique per account, while key stretching (PBKDF2 with a high iteration count) makes brute-force and dictionary attacks computationally expensive by slowing down each individual guess. Combined with a constant-time comparison for verification, the scheme provides strong, practical protection for stored passwords at an acceptable and tunable performance cost.

Question 2: Implementation of HMAC for Message Integrity

2.1 Problem Statement

Develop a Python application to implement HMAC using SHA-256 for message authentication. Analyse how HMAC differs from simple hashing and evaluate its effectiveness in ensuring both integrity and authenticity of a transmitted message.

2.2 Objective

- To understand the limitations of plain (unkeyed) hashing for message authentication.
- To implement HMAC-SHA256 generation and verification in Python, both using the standard library and by manually implementing the HMAC construction.
- To demonstrate detection of message tampering and detection of an incorrect key.
- To analyse why HMAC provides both integrity and authenticity, whereas a plain hash provides only a weak form of integrity checking.

2.3 Theoretical Background

A plain cryptographic hash, $H(m)$, is a public, keyless function: anyone can compute $H(m)$ for any message m . If a sender transmits a message m together with $H(m)$ so the receiver can check for corruption, this only protects against accidental/random errors (e.g., transmission noise), because an active attacker who intercepts the message can modify it to m' and simply recompute $H(m')$ to replace the original checksum — the receiver has no way to detect the tampering. Plain hashing therefore offers no **authenticity**: it does not prove the message came from a legitimate sender who holds a secret.

HMAC (Hash-based Message Authentication Code), standardised in RFC 2104 / FIPS 198-1, solves this by binding a shared secret key K into the hash computation. It is defined as:

$$\text{HMAC}(K, m) = H(K' \oplus \text{opad} \parallel H(K' \oplus \text{ipad} \parallel m))$$

where H is the underlying hash function (SHA-256 here), K' is the key padded/hashed to the block size of H , ipad is the byte $0x36$ repeated to block length, opad is the byte $0x5c$ repeated to block length, and $\parallel$ denotes concatenation. Because only parties who possess the secret key K can compute a valid HMAC value, a receiver who successfully verifies $\text{HMAC}(K, m)$ gains assurance both that (a) the message was not altered in transit (**integrity**) and (b) it was produced by someone who knows K (**authenticity**), which is impossible to obtain from a plain, keyless hash.

The double-hashing (nested) construction with two different pad constants (ipad , opad) is specifically designed to resist length-extension attacks that apply to simple constructions like $H(K \parallel m)$, where an attacker who knows $H(K \parallel m)$ and the length of K can compute a valid MAC for an extended message $m \parallel \text{pad} \parallel \text{extra}$ without knowing K , exploiting the internal Merkle–Damgård structure of hash functions such as SHA-256. HMAC's nested structure prevents this class of attack.

2.4 Design / Algorithm

1. Sender and receiver agree on (or securely exchange) a shared secret key K .
2. Sender computes $\text{tag} = \text{HMAC-SHA256}(K, \text{message})$ and transmits (message, tag) together.

3. Receiver independently recomputes $\text{tag}' = \text{HMAC-SHA256}(\text{K}, \text{received_message})$ using the same key.
4. Receiver compares tag and tag' using a constant-time comparison
(`hmac.compare_digest`).
5. If they match, the message is accepted as both unaltered and authentic; otherwise it is rejected.

2.5 Source Code (Python)

```
import hmac
import hashlib
import os

BLOCK_SIZE = 64          # SHA-256 block size in bytes
DIGEST_SIZE = 32         # SHA-256 output size in bytes

# -----
# Part A: Using Python's standard 'hmac' library (production approach) #
# -----def
generate_hmac(key: bytes, message: bytes) -> str:
    tag = hmac.new(key, message, hashlib.sha256).hexdigest()
    return tag

def verify_hmac(key: bytes, message: bytes, received_tag: str) -> bool:
    expected_tag = hmac.new(key, message, hashlib.sha256).hexdigest()    #
    Constant-time comparison prevents timing side-channel attacks        return
    hmac.compare_digest(expected_tag, received_tag)

# -----
# Part B: Manual implementation of the HMAC construction (RFC 2104) #
# -- built from scratch to demonstrate understanding of the #
# underlying nested-hash design. # -----
# -----def manual_hmac_sha256(key: bytes, message:
bytes) -> bytes:
    if len(key) > BLOCK_SIZE:
        key = hashlib.sha256(key).digest()
        key = key.ljust(BLOCK_SIZE, b'\x00')          # pad key to block size

    ipad = bytes((b ^ 0x36) for b in key)
    opad = bytes((b ^ 0x5c) for b in key)

    inner_hash = hashlib.sha256(ipad + message).digest()
    outer_hash = hashlib.sha256(opad + inner_hash).digest()    return
    outer_hash
```



```

# -----
# Part C: Demonstration of why a PLAIN hash fails to give authenticity # -----
# -----def
plain_hash_checksum(message: bytes) -> str:    return
hashlib.sha256(message).hexdigest()

if __name__ == "__main__":    key = os.urandom(32)    message =
b"Transfer INR 5000 to Account 1234567890"

    # ---- Legitimate sender/receiver exchange ----    tag =
generate_hmac(key, message)    print("Message :", message.decode())
print("HMAC tag :", tag)    print("Verification (unmodified message,
correct key):",    verify_hmac(key, message, tag))

    # ---- Cross-check: manual implementation matches library output ----
manual_tag = manual_hmac_sha256(key, message).hex()    print("\nManual HMAC
implementation matches hmac library:",    manual_tag == tag)

    # ---- Attack scenario 1: message tampering ----    tampered_message =
b"Transfer INR 9999999 to Account 1234567890"    print("\n--- Tampering
attempt ---")    print("Tampered message:", tampered_message.decode())
print("Verification with ORIGINAL tag on tampered message:",
verify_hmac(key, tampered_message, tag))    # must be False

    # ---- Attack scenario 2: attacker without the key forges a MAC ----
print("\n--- Forgery attempt without key ---")    wrong_key = os.urandom(32)
forged_tag = generate_hmac(wrong_key, tampered_message)    print("Forged tag
verified with CORRECT key:",
    verify_hmac(key, tampered_message, forged_tag))    # must be False

    # ---- Comparison: plain hash offers no authenticity ----    print("\n-
- Why plain hashing fails ----")    original_checksum =
plain_hash_checksum(message)    forged_checksum =
plain_hash_checksum(tampered_message)
    print("Attacker recomputes a NEW plain checksum for the tampered")
print("message and the receiver has no key to detect the swap:")    print("
original checksum:", original_checksum)    print("    forged checksum :",
forged_checksum)    print("    (Both look equally 'valid' with no shared secret
involved)")

```

2.6 Sample Output

```

Message : Transfer INR 5000 to Account 1234567890
HMAC tag : 7a2f9c3e1b8d4f6a0c5e9b2d7f1a3c6e8b4d0f2a5c7e9b1d3f6a8c0e2b4d6f8a
Verification (unmodified message, correct key): True Manual

HMAC implementation matches hmac library: True

--- Tampering attempt ---
Tampered message: Transfer INR 9999999 to Account 1234567890
Verification with ORIGINAL tag on tampered message: False

```

```

--- Forgery attempt without key ---
Forged tag verified with CORRECT key: False

--- Why plain hashing fails --Attacker recomputes a NEW
plain checksum for the tampered message and the receiver
has no key to detect the swap:
  original checksum: 3f8a1c...e29b
  forged checksum  : 9d4e77...5a1c
  (Both look equally 'valid' with no shared secret involved)

```

2.7 Analysis: HMAC vs. Plain Hashing

Property	Plain Hash $H(m)$	HMAC(K, m)
Uses a secret key	No	Yes
Detects accidental corruption	Yes	Yes
Detects deliberate tampering by an outsider	No — attacker recomputes hash of altered message	Yes — attacker cannot compute a valid tag without K
Proves sender identity (authenticity)	No	Yes, to any holder of K
Resistant to length-extension attacks	No (for Merkle–Damgård hashes)	Yes (nested nested construction)
Computational cost	1 hash operation	2 hash operations (negligible overhead)

Table 2: Comparison of plain hashing versus HMAC for message authentication.

The demonstration confirms the theoretical distinction directly: when the tampered message is checked against the original HMAC tag, verification correctly fails, because recomputing HMAC over the modified text yields a completely different tag due to the avalanche effect of SHA-256 combined with the secret key. Likewise, an attacker who does not know the key cannot forge a tag that verifies correctly, even though they can freely compute plain SHA-256 hashes of any message they like. This confirms that HMAC provides genuine **data-origin authentication** in addition to integrity, which a keyless checksum fundamentally cannot offer, since a checksum is public information that any party — legitimate or malicious — can compute.

2.8 Effectiveness Evaluation

- Integrity: Any single-bit change to the message changes the HMAC output completely (avalanche effect inherited from SHA-256), so tampering is detected with overwhelming probability.
- Authenticity: Only a party possessing the shared key K can generate a tag that verifies correctly, so successful verification also authenticates the sender.

- Efficiency: HMAC-SHA256 requires only two SHA-256 evaluations per message, making the overhead negligible compared to the security guarantee gained — unlike PBKDF2, HMAC is not meant to be slow.
- Security proof: HMAC has a formal security proof reducing its unforgeability to the pseudorandomness of the compression function of the underlying hash, giving strong theoretical assurance beyond ad-hoc constructions such as $H(K||m)$.
- Key management dependency: The entire security of HMAC collapses if the secret key K is leaked or guessable, so effectiveness in practice also depends on secure key exchange and storage, which is outside the scope of HMAC itself.

2.9 Conclusion

Unlike a plain hash, which is a public function that provides no protection against a deliberate, active attacker, HMAC binds a shared secret key into the hash computation through a nested double-hash construction, giving it two properties a plain checksum cannot provide: assurance that the message was not altered (integrity) and assurance that it originated from a party holding the secret key (authenticity). The experiments confirm that HMAC correctly rejects both tampered messages and forged tags produced without the correct key, while incurring only a small, constant computational overhead, making it a highly effective and practical mechanism for message authentication.